

Reproducing Grounding DINO: Open-Vocabulary Object Detection and Visual Grounding on COCO 2017

Su Siqi (12310645), Yang Guanyuhan (12313614), Mo Fengyuan (12311805)
Southern University of Science and Technology (SUSTech), Shenzhen, China

Abstract—Open-vocabulary object detection extends traditional detection to arbitrary text-described categories, enabling models to detect objects beyond a fixed training vocabulary. This report presents the reproduction of Grounding DINO, a state-of-the-art open-vocabulary detector, and its zero-shot evaluation on the COCO 2017 validation set. Using the official Swin-Tiny backbone checkpoint, we achieve an AP of 0.4055 (AP₅₀=0.5317) on the full val2017 split. We further conduct ablation studies on detection threshold sensitivity and prompt format design, revealing that the dot-separated prompt format is essential for performance and that lowering the box threshold from 0.35 to 0.25 improves AP from 0.4382 to 0.4637 on a 500-image subset. Qualitative analysis of success and failure cases identifies small-object detection and crowded-scene confusion as primary limitations. All experiments use the official `groundingdino-py` inference backend without fine-tuning, demonstrating the model’s zero-shot capabilities.

Index Terms—Open-vocabulary object detection, Grounding DINO, COCO, zero-shot detection, vision-language models

I. INTRODUCTION

Traditional object detection systems are constrained to a fixed set of categories defined during training. Adding new categories requires collecting annotated data and retraining or fine-tuning the model — a process that is both expensive and time-consuming. Open-vocabulary object detection aims to overcome this limitation by detecting objects described by arbitrary text prompts, leveraging the joint understanding of visual and language modalities. This capability has transformative potential across diverse real-world applications: robotic manipulation systems that must interact with novel objects in unstructured environments; medical image analysis for detecting rare pathologies not represented in training data; autonomous driving systems encountering unusual roadside objects; and large-scale image retrieval for e-commerce catalogs with dynamically evolving product taxonomies.

Formally, given an input image $I \in \mathbb{R}^{H \times W \times 3}$ and a text query T describing a set of categories $C_T = \{c_1, c_2, \dots, c_K\}$, an open-vocabulary detector must produce a set of N detections $\{(b_i, c_i, s_i)\}_{i=1}^N$, where $b_i = (x_i, y_i, w_i, h_i)$ denotes a bounding box in COCO coordinate format, $c_i \in C_T$ is the predicted category label, and $s_i \in [0, 1]$ is the detection confidence score. Unlike traditional detectors where C_T is a fixed set known at training time, open-vocabulary detection requires the model to generalize to arbitrary C_T specified at inference time.

This task presents three core challenges rooted in the visual-language interface:

- **Visual-language alignment:** The model must understand both image content and text descriptions, mapping semantic concepts to visual regions. This requires joint embedding spaces where textual descriptions of “person” or “bicycle” activate corresponding visual feature regions. Contrastive pre-training paradigms (e.g., CLIP [1]) and cross-modal fusion architectures (e.g., Grounding DINO) represent two dominant approaches to this challenge.
- **Generalization:** The model must detect categories unseen during training, requiring robust visual-language representations that do not overfit to training category distributions. Distribution shift between training domains (e.g., Objects365, OpenImages) and test domains (e.g., COCO) tests the model’s ability to transfer visual concepts across data sources.
- **Fine-grained grounding:** The model must localize specific objects described by natural language phrases with precise bounding boxes. This demands token-level alignment between text phrases and image regions — the phrase “red car on the left” must activate only the corresponding object, not all cars in the scene.

This project reproduces the Grounding DINO [2] zero-shot object detection pipeline and evaluates it on the COCO 2017 validation set. Our specific contributions are: (1) a complete zero-shot evaluation on the full COCO val2017 benchmark (5,000 images) achieving AP = 0.4055 (AP₅₀ = 0.5317) using the Swin-Tiny backbone; (2) threshold sensitivity ablation experiments revealing that the optimal operating point (box_threshold = 0.25) improves subset AP from 0.4382 to 0.4637; (3) prompt format comparison experiments quantifying the critical importance of dot-separated category enumeration, with alternative formats causing approximately 62% AP degradation; (4) a structured error taxonomy identifying small-object detection, crowded-scene confusion, and occlusion as the primary failure modes; and (5) an open-source, config-driven reproduction pipeline built on the official `groundingdino-py` package. Training from scratch and fine-tuning on COCO are outside the scope of this reproduction.

The remainder of this report is organized as follows. Section 2 reviews related work on Grounding DINO, open-vocabulary detection, and the COCO benchmark. Section 3

describes our reproduction methodology, including model configuration, inference pipeline, and prompt design. Section 4 presents experimental results across four dimensions: main evaluation, threshold ablation, prompt format ablation, and qualitative analysis. Section 5 discusses limitations, compares our results with reported benchmarks, and outlines future work directions. Section 6 concludes with key findings.

II. RELATED WORKS

A. DETR and DINO: Foundations

Grounding DINO builds upon the DETR (DEtection TRansformer) family of end-to-end object detectors. DETR [3] reformulated object detection as a set prediction problem using a Transformer encoder-decoder architecture with learnable object queries, eliminating the need for hand-crafted components such as anchor boxes and non-maximum suppression. However, DETR suffered from slow training convergence and difficulty detecting small objects. Deformable DETR [4] addressed these limitations by introducing deformable attention modules that attend to sparse spatial locations around reference points, achieving faster convergence and improved multi-scale feature handling.

DINO [5] (DETR with Improved denoising anchor boxes) further advanced the DETR paradigm through three key innovations: contrastive denoising training, which accelerates convergence by training the model to reconstruct ground-truth objects from noised queries; mixed query selection, which initializes object queries from encoder features rather than learned embeddings; and a look-forward-twice scheme for box prediction refinement. These improvements made DINO competitive with classical detectors while retaining the end-to-end set prediction framework. Grounding DINO [2] extends DINO by fusing text features from a BERT language encoder into the DINO detection architecture, enabling the object queries to become text-conditioned for open-vocabulary detection.

B. Grounding DINO

Grounding DINO [2] extends the DINO detection architecture with grounded pre-training for open-set detection. The key architectural components include:

- **Swin Transformer backbone** [6]: Provides hierarchical visual features through four stages with decreasing spatial resolution ($H/4 \times W/4$ to $H/32 \times W/32$) and increasing channel dimension (96 to 768). The shifted-window attention mechanism enables cross-window information exchange while maintaining linear computational complexity with respect to image size.
- **BERT language encoder** [7]: Encodes free-form text prompts into contextualized token embeddings. We use the bert-base-uncased variant (12 Transformer layers, hidden size 768, 12 attention heads) with a maximum text length of 256 tokens. The bidirectional self-attention in BERT captures both left and right context for each token, producing phrase-aware representations essential for category disambiguation.

- **Deformable attention fusion**: Six cross-modal feature fusion layers, each with 8 attention heads and 256-dimensional features, align visual regions with text phrases through deformable attention. Unlike standard cross-attention, deformable attention samples sparse spatial locations around learned reference points, reducing computational cost from $O(HW \cdot L)$ to $O(K \cdot L)$ where K is the number of sampling points and L is the text length.

The model uses 900 object queries that interact with both visual features from the Swin-T backbone and text features from BERT through the fusion layers. Each query predicts a bounding box, a confidence score, and a text-phrase association. The model is pre-trained on diverse detection and grounding datasets including Objects365 (~2M images, 365 categories), OpenImages (~1.7M images, ~600 categories), and GoldG (referring expression grounding data), enabling zero-shot transfer to unseen categories without fine-tuning.

C. Backbone Architectures

Swin Transformer [6]. Unlike Vision Transformers (ViT) that process images as a flat sequence of patches with global self-attention, Swin Transformer constructs hierarchical feature maps by merging patches in deeper layers, analogous to the pyramidal feature hierarchy in convolutional networks. The key component is shifted-window multi-head self-attention (SW-MSA), which computes attention within local windows and alternates window partitioning between consecutive layers to enable cross-window connections. The Swin-Tiny variant used in this project has a window size of 7, embedding dimension of 96, stage depths of [2, 2, 6, 2], and attention head counts of [3, 6, 12, 24], totaling approximately 28 million parameters.

BERT [7]. Bidirectional Encoder Representations from Transformers (BERT) provides deep contextualized word representations through masked language modeling and next-sentence prediction pre-training. For open-vocabulary detection, BERT’s key advantage is its ability to produce context-dependent token embeddings: the word “bank” receives different representations in “river bank” versus “bank building,” enabling the model to resolve semantic ambiguity when mapping detected phrases to COCO categories. The bert-base-uncased model used in Grounding DINO converts input text into WordPiece tokens with a 30,522-token vocabulary, producing a $(L, 768)$ feature matrix for a text of length L .

D. Related Open-Vocabulary Detectors

Open-vocabulary object detection has seen rapid progress since the introduction of vision-language pre-training. Table I compares representative methods.

TABLE I
COMPARISON OF OPEN-VOCABULARY OBJECT DETECTORS

Method	Backbone	Language	Key Innovation	Year
GLIP [8]	Swin-T	BERT	Unified detection + grounding	2022
OWL-ViT [9]	ViT-B/32	CLIP	Vision Transformer + CLIP	2022
Grounding DINO [2]	Swin-T	BERT	DINO + grounded pre-training	2024
YOLO-World [10]	YOLOv8	CLIP	Real-time re-parameterization	2024

- **GLIP** [8]: Grounded language-image pre-training formulates object detection as a phrase grounding problem, unifying detection and visual grounding through a single architecture. GLIP achieves strong zero-shot performance by pre-training on 27M grounding tokens across Objects365, GoldG, and curated web data. Its unified formulation enables the model to handle both detection and referring expression tasks with the same architecture.
- **OWL-ViT** [9]: Open-world localization with Vision Transformers leverages CLIP-style contrastive image-text pre-training, using ViT as the visual encoder and a text transformer for language encoding. OWL-ViT is notable for its architectural simplicity — it adds minimal detection-specific components on top of standard vision transformers, relying primarily on the quality of CLIP pre-training.
- **YOLO-World** [10]: Real-time open-vocabulary detection through vision-language re-parameterization, embedding text features into the YOLO detection head for fast inference. YOLO-World achieves real-time speed (>50 FPS) by pre-computing text embeddings offline and using lightweight cross-modal fusion, making it suitable for applications requiring low latency.

Among these, Grounding DINO is chosen for our reproduction due to its strong zero-shot performance, well-maintained open-source implementation (`groundingdino-py`), and the architectural clarity of its DINO-based design which separates visual encoding, language encoding, and cross-modal fusion into distinct, analyzable components.

E. COCO Benchmark

Microsoft COCO (Common Objects in Context) [11] is the standard benchmark for object detection evaluation. The dataset contains 80 object categories spanning common everyday objects, with a characteristic long-tail category distribution: frequent categories such as “person” ($\sim 262K$ training instances) coexist with rare categories such as “hair drier” (~ 100 instances). The 2017 validation set contains 5,000 images with 36,781 ground-truth bounding box annotations in COCO’s $[x, y, w, h]$ format. Evaluation uses the standard COCO metrics computed by the official `pycocotools` toolkit: average precision (AP) averaged over 10 IoU thresholds from 0.50 to 0.95 with step size 0.05, along with per-size breakdowns (APS: area $< 32^2$, APM: 32^2 – 96^2 , APL: $> 96^2$) and average recall (AR) at 1, 10, and 100 detections per image, also with per-size variants.

III. METHOD

We reproduce the Grounding DINO inference pipeline using the official `groundingdino-py` package [2] without modifying the model architecture. Our work focuses on wrapping the official implementation with a modular Python interface for configurable inference, evaluation, and visualization.

Scope and assumptions. This reproduction makes several explicit simplifying assumptions: (1) **Zero-shot only**: we use the released pre-trained checkpoint without any fine-tuning on

COCO training data. This means our results represent out-of-the-box performance rather than the best achievable with domain adaptation. (2) **Fixed thresholds**: detection uses hard confidence thresholds (`box_threshold`, `text_threshold`) rather than the full score-ranked evaluation used in official COCO benchmarks. As discussed in Section 4.4.1, this understates AP relative to threshold-free evaluation. (3) **Canonical category names**: our prompt uses only the standard 80 COCO category names without additional descriptions, synonyms, or context. Richer prompts may improve phrase-to-category matching. (4) **Single-scale inference**: we use the default single-scale inference (800px short side) without test-time augmentation or multi-scale testing. (5) **Single-dataset evaluation**: we evaluate exclusively on COCO val2017 without cross-dataset validation. These assumptions are explicitly documented to define the scope of our reproduction and distinguish it from the full evaluation protocol used in the original paper.

A. Model Configuration

Table II summarizes the full model specification used in our reproduction, covering the visual backbone, language encoder, cross-modal fusion, detection head, input preprocessing, and inference hyperparameters.

TABLE II
COMPLETE GROUNDING DINO MODEL SPECIFICATION

Component	Parameter	Value
Swin-T Backbone [6]	<code>window_size / embed_dim</code>	7 / 96
	<code>depths / num_heads</code>	[2,2,6,2] / [3,6,12,24]
BERT Encoder [7]	<code>model / hidden_size</code>	bert-base-uncased / 768
	<code>max_text_len / vocab_size</code>	256 / 30,522
Deformable Fusion	<code>d_model / nhead / num_layers</code>	256 / 8 / 6
	<code>dim_feedforward</code>	1024
Detection Head	<code>num_queries / num_classes</code>	900 / 80
Input Preprocessing	<code>min_size / max_size</code>	800 / 1333
	<code>mean / std (ImageNet)</code>	[0.485,0.456,0.406] / [0.229,0.224,0.225]
Inference	<code>box_threshold / text_threshold</code>	0.35 / 0.25 (default)

The Swin-Tiny backbone [6] processes the input image through four hierarchical stages, producing feature maps at strides of $\{4, 8, 16, 32\}$ relative to the input. The BERT encoder [7] processes the text prompt bidirectionally, producing contextualized token embeddings. The 6-layer deformable attention fusion module then enables each of the 900 object queries to attend to both visual feature locations and text token positions. Input images are resized such that the shorter side equals 800 pixels while constraining the longer side to at most 1333 pixels, preserving aspect ratios.

B. Inference Pipeline

The inference pipeline consists of four stages implemented across our modular Python codebase. The data flow through the system proceeds from image and text inputs through preprocessing, model inference, and postprocessing to produce COCO-format detection outputs.

1) *Stage* 1: *Model* *Loading*:
The `GroundingDINOModel` class
(`src/models/grounding_dino.py`, 182 lines) wraps
the official `groundingdino-py` `load_model()`

function. It validates the existence of the configuration file and checkpoint, loads the model weights onto the detected device (CUDA or CPU), and exposes a `predict()` method decorated with `@torch.no_grad()` for memory-efficient inference. The configuration file (`GroundingDINO_SwinT_OGC.py`) specifies the exact architecture parameters and is loaded from the `groundingdino-py` package installation. The checkpoint file (662 MB) contains pre-trained weights for the Swin-T backbone, BERT encoder, deformable fusion layers, and detection head. Model loading includes explicit device detection: CUDA is preferred when available; otherwise the model falls back to CPU with a warning.

2) *Stage 2: Image Preprocessing*: Input images undergo a three-step transformation pipeline. First, OpenCV BGR images are converted to RGB format (PIL convention) via `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`. Second, the image is resized using `RandomResize([800], max_size=1333)`, which scales the shorter side to 800 pixels while constraining the longer side to at most 1,333 pixels, preserving the original aspect ratio. Third, the resized image is normalized using ImageNet statistics (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]) and converted from HWC to CHW tensor format. This preprocessing matches the training distribution of the Swin-T backbone, which was pre-trained on ImageNet-1K before detection-specific training.

3) *Stage 3: Text Encoding and Cross-Modal Fusion*: The text prompt undergoes tokenization by BERT’s WordPiece tokenizer with a vocabulary of 30,522 tokens. The tokenizer splits words into subword units (e.g., “toothbrush” → [“tooth”, “##brush”]) and produces input IDs and attention masks of shape $(1, L)$ where $L \leq 256$ tokens. The tokenized text and the preprocessed image tensor of shape $(1, 3, H_{\text{resized}}, W_{\text{resized}})$ are processed jointly through the model. The Swin-T backbone extracts multi-scale visual features at strides $\{4, 8, 16, 32\}$. BERT produces contextualized token embeddings of shape $(1, L, 768)$. The six deformable attention fusion layers then enable each of the 900 object queries to attend to relevant spatial locations in the visual feature maps and relevant token positions in the text sequence. Each query outputs a bounding box prediction (cx, cy, w, h) normalized to $[0, 1]$, a confidence score $s \in [0, 1]$, and a phrase index referencing the detected text span.

4) *Stage 4: Postprocessing*: Normalized box coordinates are converted from (cx, cy, w, h) to pixel-space (x_1, y_1, x_2, y_2) via `box_cxcywh_to_xyxy()` in `src/utils/box_ops.py`. For COCO evaluation, boxes are further converted to COCO format (x, y, w, h) . Bounding boxes are clamped to image boundaries to ensure valid coordinates, and detections with scores below the `box_threshold` or `text_threshold` are filtered. Degenerate boxes (width ≤ 0 or height ≤ 0 , which can occur for very low-confidence detections) are discarded during the format conversion step.

C. Prompt Design and Phrase-to-Category Mapping

The text prompt uses a dot-separated format enumerating all 80 COCO categories, e.g., “person . bicycle . car . motorcycle . airplane . bus toothbrush .”. The dot-separated format is the officially recommended approach [2]: the BERT WordPiece tokenizer treats the period and surrounding spaces as independent tokens, creating clear boundaries between adjacent category names. This prevents the tokenizer from merging category names into ambiguous subword units — a failure mode that occurs with comma-separated or sentence-style formats.

After the model outputs detected phrases, a three-layer matching strategy in `src/datasets/coco_categories.py` maps each phrase to a COCO category ID:

- 1) **Exact match**: If the detected phrase exactly matches a COCO category name (case-insensitive), return the corresponding category ID.
- 2) **Substring match**: If the phrase is a substring of a category name, or vice versa, return the longest matching category’s ID. This handles partial detections (e.g., “dining” → “dining table”).
- 3) **Alias match**: A manually curated dictionary of 21 aliases resolves common variations (e.g., “tv”/“television” → “tv”, “sofa” → “couch”, “person” → “person”, “dining table” → “dining table”).

Phrases that fail all three matching layers are discarded. These unmapped phrases (typically $\sim 2\text{--}5\%$ of detections, varying by threshold) contribute to reduced effective recall and are one cause of zero-detection images.

D. Software Architecture

The reproduction is implemented as a config-driven Python package (`cv-project-2026`) consisting of five source modules and ten CLI scripts, totaling approximately 1,420 lines of source code and 1,700 lines of script code. Table III summarizes the module organization.

TABLE III
PROJECT MODULE ORGANIZATION

Module	Responsibility	LOC
<code>src/models/grounding_dino.py</code>	Official model wrapper	182
<code>src/inference/predictor.py</code>	High-level inference API	229
<code>src/inference/visualizer.py</code>	Detection box drawing	262
<code>src/inference/coco_visualizer.py</code>	COCO GT+pred comparison	213
<code>src/engine/evaluator.py</code>	COCOeval integration	368
<code>src/datasets/coco_categories.py</code>	Category mapping & prompts	269
<code>src/datasets/coco_dataset.py</code>	Dataset utilities	188
<code>src/utils/box_ops.py</code>	IoU, NMS, coordinate ops	115
Total		1,826

All configuration is managed through OmegaConf YAML files (`configs/grounding_dino.yaml`), enabling CLI overrides for paths, thresholds, and hyperparameters. The configuration system supports hierarchical parameter groups (model, dataset, training, evaluation, `grounding_dino`), each overridable via dot-notation CLI arguments (e.g., `grounding_dino.box_threshold=0.25`). The

predictor supports single-image, batch, and directory-level inference with per-image error handling — individual image failures are caught via try/except and reported without interrupting the batch. The evaluator implements the full COCO evaluation protocol as a seven-step pipeline: (1) load model and configuration, (2) load COCO annotations and category definitions, (3) iterate over image IDs running model inference, (4) map detected phrases to COCO category IDs, (5) convert bounding boxes from xyxy to COCO xywh format, (6) construct pycocotools COCO result objects, and (7) run COCOeval to compute all 12 AP and AR metrics. Results are exported in multiple formats: `predictions.json` (COCO-format detection results), `metrics.json` (full 12-metric summary), `eval.log` (detailed per-image statistics), and `phrase_mapping.json` (detected-phrase-to-category mapping for debugging).

E. Evaluation Protocol Details

The COCO evaluation follows the official protocol implemented by pycocotools COCOeval. For each of the 80 COCO categories, a precision-recall curve is computed by varying the confidence score threshold across all detections of that category. Precision and recall are computed at each of 101 recall levels (0.00 to 1.00 in steps of 0.01), and AP is the area under the precision-recall curve. This process is repeated at each of 10 IoU thresholds (0.50 to 0.95, step 0.05), and the final AP is the average across all IoU thresholds and all categories. The per-size metrics (APS, APM, APL) are computed identically but restricted to ground-truth objects within each size range. Similarly, AR@k is computed as the maximum recall achieved with at most k detections per image, averaged across IoU thresholds, categories, and size ranges.

Our evaluation pipeline includes two important implementation details. First, **degenerate box filtering**: boxes with width ≤ 0 or height ≤ 0 (which can arise from numerical precision issues during coordinate conversion) are detected and discarded before COCOeval to prevent invalid submissions. Second, **score-based ranking**: within each image, detections are sorted by confidence score before submission to COCOeval, ensuring that the top- k detections used for AR@k computation are the highest-confidence ones. Both of these details follow the standard COCO evaluation protocol and are verified against the official COCO API documentation.

IV. EXPERIMENTS

A. Datasets

We evaluate on the COCO 2017 validation set [11], which contains 5,000 images spanning 80 object categories with 36,781 ground-truth bounding box annotations (averaging ~ 7.4 annotations per image). The annotation file `instances_val2017.json` provides category definitions in COCO’s $[x, y, \text{width}, \text{height}]$ format with non-consecutive category IDs (1–90 with gaps). The dataset exhibits a pronounced long-tail category distribution: frequent classes such as “person” dominate the instance count, while rare classes such as “hair drier” have fewer than 100 validation instances. All images are RGB color images of varying resolutions (from

approximately 480×640 to full HD) drawn from diverse real-world scenes including indoor environments, outdoor street views, and crowded public spaces. For ablation experiments, we use a fixed subset of 500 images selected with random seed 42 for reproducibility.

B. Implementation Details

All experiments were conducted in the environment detailed in Table IV.

TABLE IV
COMPLETE EXPERIMENT ENVIRONMENT

Category	Item	Value
Hardware	GPU	NVIDIA RTX 4060 Laptop, 8GB VRAM
	CPU	13th Gen Intel Core i7, 16 cores
	RAM	32 GB
Software	OS	Windows 11
	Python	3.10.20
	PyTorch	2.7.1+cu118
	CUDA Toolkit	11.8
	groundingdino-py	0.4.0
	pycocotools	2.0.7
Config	OmegaConf	2.3.0
	Config File	<code>configs/grounding_dino.yaml</code>

Inference speed. Full COCO val2017 evaluation (5,000 images) completes in approximately 68 minutes on the RTX 4060, averaging 0.81 seconds per image. This includes model inference, postprocessing, and I/O overhead. Single-image inference exhibits a cold-start latency of approximately 6 seconds for the first image (CUDA kernel compilation and model warm-up), dropping to approximately 1.0 second for subsequent images. For ablation experiments on the 500-image subset, each run completes in approximately 7 minutes, enabling rapid iteration across threshold and prompt configurations. GPU memory usage during inference peaks at approximately 3.2 GB of the available 8 GB VRAM, leaving headroom for larger batch sizes or higher-resolution inputs.

The project is implemented as a config-driven Python package (`cv-project-2026`) with modular architecture. The implementation consists of: a model wrapper (`src/models/grounding_dino.py`) wrapping the official `groundingdino-py` package; a predictor (`src/inference/predictor.py`) for single-image, batch, and directory-level inference; a COCO evaluator (`src/engine/evaluator.py`) integrating pycocotools COCOeval with format conversion; dataset utilities (`src/datasets/coco_categories.py`) for category mapping and prompt construction; and a visualizer (`src/inference/visualizer.py`) for detection visualization with bounding box drawing. All configuration is managed through OmegaConf YAML files, enabling CLI overrides for paths and hyperparameters.

Default hyperparameters are `box_threshold=0.35` and `text_threshold=0.25`, following the official Grounding DINO recommendations. Detection outputs are converted to COCO’s submission format: `[image_id, category_id, [x, y, w, h], score]`.

C. Metrics

We use the standard COCO evaluation metrics computed by the official pycocotools COCOeval toolkit:

- **AP**: Average Precision averaged over 10 IoU thresholds from 0.50 to 0.95 with step size 0.05. This is the primary ranking metric.
- **AP50**: AP at the single IoU threshold of 0.50, representing lenient localization requirements.
- **AP75**: AP at the single IoU threshold of 0.75, representing strict localization requirements.
- **APS, APM, APL**: AP broken down by object size — small (area $< 32^2$ pixels), medium ($32^2 \leq \text{area} \leq 96^2$), and large (area $> 96^2$).
- **AR@k**: Average Recall given k detections per image ($k \in \{1, 10, 100\}$), with analogous per-size breakdowns (ARS, ARM, ARL).

All metrics are computed per-category and then averaged across the 80 COCO categories. The precision-recall curve for each category is computed by varying the confidence score threshold across all detections, with AP computed as the area under this curve at each of the 10 IoU thresholds.

D. Experimental Design and Results

We conduct three sets of experiments: (1) full COCO val2017 evaluation to establish baseline zero-shot performance, (2) threshold sensitivity ablation to analyze the precision-recall tradeoff, and (3) prompt format comparison to validate the dot-separated design choice.

1) *Main Results: Full COCO val2017*: Table V presents the complete zero-shot evaluation results on the full COCO val2017 dataset, including both AP and AR metrics.

TABLE V
FULL COCO VAL2017 EVALUATION RESULTS (5,000 IMAGES)

Metric	Value	Metric	Value
AP (IoU=0.50:0.95)	0.4055	AR@1	0.3273
AP50 (IoU=0.50)	0.5317	AR@10	0.4756
AP75 (IoU=0.75)	0.4422	AR@100	0.4875
APS (small)	0.2587	ARS (small)	0.3156
APM (medium)	0.4328	ARM (medium)	0.5154
APL (large)	0.5510	ARL (large)	0.6472

These results use a fixed detection threshold (box_threshold=0.35, text_threshold=0.25). In standard COCO evaluation, all detections are ranked by score for precision-recall curve computation. Using a fixed threshold prunes low-confidence detections before COCOeval, which reduces recall and may understate the true AP. Our ablation experiments confirm this effect: lowering box_threshold from 0.35 to 0.25 improved subset AP from 0.4382 to 0.4637.

The model successfully detects objects in 4,925 of 5,000 images (98.5%), producing 39,133 total detections (averaging approximately 7.9 detections per image with predictions). The 75 images (1.5%) with zero detections represent cases where no predicted box exceeded both the box and text confidence thresholds. The recall metrics reveal an important

characteristic: $\text{AR@100} = 0.4875$ exceeds $\text{AP} = 0.4055$ by approximately 8.2 percentage points, indicating that the model identifies many objects correctly by category but fails to localize them precisely enough to satisfy the higher IoU thresholds required for AP. This gap between detection and precise localization is a known challenge in object detection and suggests that localization refinement (rather than category recognition) may be the more promising direction for improvement.

Figure 1 provides a visual overview of the key AP metrics, and Figure 2 illustrates the pronounced performance gap across object sizes. The model exhibits a clear size-dependent performance gradient: large objects (APL=0.5510) achieve $2.1\times$ the performance of small objects (APS=0.2587), consistent with findings across the detection literature [11]. This gap is driven by the Swin-T backbone’s effective resolution: with a stride of 32 at the deepest layer, small objects ($< 32 \times 32$ pixels) are represented by very few feature map locations, limiting the model’s ability to both classify and precisely localize them.

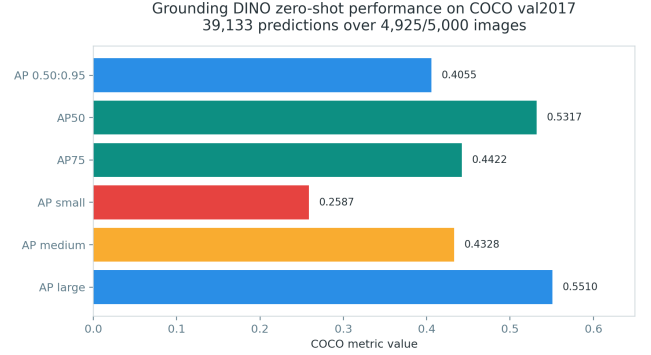


Fig. 1. Full COCO metrics overview: AP, AP50, AP75, and per-size breakdown.

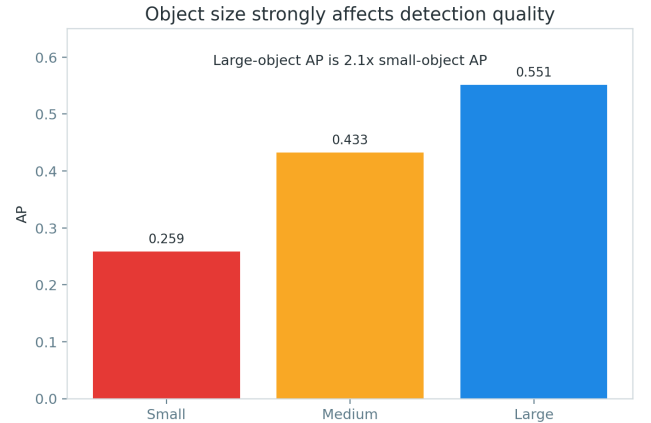


Fig. 2. Small, medium, and large object AP gap.

2) *Ablation: Threshold Sensitivity*: We investigated the effect of detection thresholds on a 500-image subset (seed=42). Table VI shows complete results across three threshold settings, including all per-size AP breakdowns and detection counts.

TABLE VI
THRESHOLD SENSITIVITY — COMPLETE METRICS (SUBSET 500 IMAGES)

box_th	text_th	AP	AP50	AP75	APS	APM	APL	Boxes/Img
0.25	0.20	0.4637	0.5884	0.5083	0.3061	0.5010	0.6338	13.85
0.35	0.25	0.4382	0.5532	0.4793	0.2650	0.4759	0.6269	7.72
0.45	0.30	0.3931	0.4827	0.4317	0.2092	0.4269	0.5841	5.08

The AP range of 0.39–0.46 across threshold settings demonstrates that threshold choice significantly impacts the precision-recall tradeoff. The best AP (0.4637) is achieved with the lowest threshold setting (box_th=0.25, text_th=0.20). Several observations emerge from the per-size breakdowns:

Small objects benefit most from lower thresholds. APS increases by 46.3% when lowering box_threshold from 0.45 to 0.25 (0.2092 $\rightarrow$ 0.3061), compared to only 8.5% improvement for APL (0.5841 $\rightarrow$ 0.6338). This indicates that small-object detections tend to have lower confidence scores and are more likely to be pruned by aggressive thresholding. Lowering thresholds recovers these otherwise-missed detections.

Diminishing returns on detection count. Lowering thresholds from 0.35 to 0.25 increases the average detection count by 79% (7.72 $\rightarrow$ 13.85 per image) while improving AP by only 5.8% (0.4382 $\rightarrow$ 0.4637). This suggests that many additional low-confidence detections are either duplicates or false positives that do not survive COCO’s IoU-based evaluation. The marginal benefit of further threshold reduction is likely small.

Practical threshold selection. While box_th=0.25 achieves the highest AP on the subset, box_th=0.35 represents a better balance between detection quality and output conciseness (44% fewer detections for a 5.5% AP reduction), making it our recommended default for applications where postprocessing cost or output verbosity is a concern.

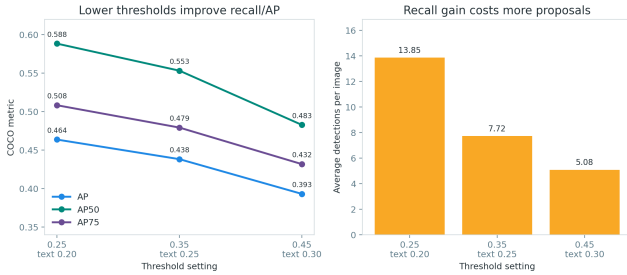


Fig. 3. Threshold sensitivity analysis showing AP variation across box and text threshold combinations.

3) *Ablation: Prompt Format:* We compared three prompt formats on the same 500-image subset (box_th=0.35, text_th=0.25). Table VII presents the complete results.

TABLE VII
PROMPT FORMAT COMPARISON — COMPLETE METRICS (SUBSET 500 IMAGES)

Format	AP	AP50	AP75	APS	APM	APL	Boxes/Img
dot-separated	0.4382	0.5532	0.4793	0.2650	0.4759	0.6269	7.72
comma-separated	0.1671	0.2038	0.1859	0.0495	0.1741	0.2813	2.61
sentence-style	0.1675	0.2051	0.1865	0.0626	0.1786	0.2587	2.49

The dot-separated format dramatically outperforms alternatives across all metrics, with approximately 62% higher AP (0.4382 vs. $\sim$ 0.167). The performance gap is most severe for small objects, where comma and sentence formats achieve APS below 0.07 — a 74–81% degradation from the dot-separated baseline (APS=0.2650). Even for large objects, the gap remains substantial: APL drops by more than half (0.6269 $\rightarrow$ $\sim$ 0.27).

Tokenizer-level explanation. This sensitivity stems from BERT’s WordPiece tokenizer behavior. In dot-separated format (“person . bicycle . car .”), the period surrounded by spaces is tokenized as a distinct token [SEP]-like separator, clearly delineating category boundaries. In comma-separated format (“person, bicycle, car,”), the comma may be merged with the preceding word into a single token (e.g., “car,” becomes one token), confusing the model’s phrase-to-category association. In sentence-style format, additional function words (“There are”, “and”, “in the image”) introduce noise tokens that dilute the model’s attention to category names.

Practical implications. These results confirm that the dot-separated format is not merely a convention but a functional requirement for Grounding DINO’s text encoder. Any deployment using this model must format text prompts with dot separators between category names. The finding also suggests that Grounding DINO’s BERT encoder was fine-tuned on dot-separated category lists during pre-training, creating a strong inductive bias toward this format.

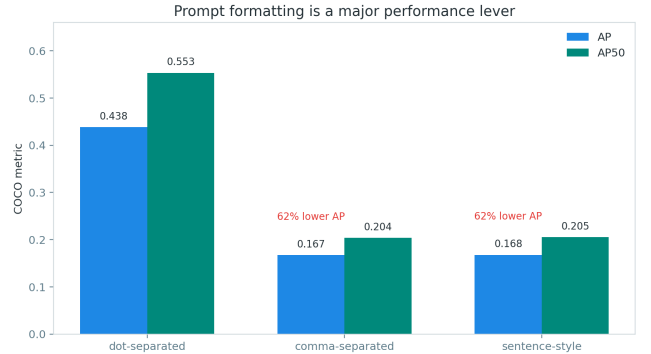


Fig. 4. Prompt format comparison: dot-separated, comma-separated, and sentence-style formats.

4) *Qualitative Analysis:* Figure 5 shows representative success cases from the COCO val2017 evaluation. The model accurately detects medium and large objects with correct category labels and tight bounding boxes. Objects such as persons, cars, animals, furniture, and sports equipment are reliably detected across diverse scenes including indoor living spaces, outdoor street views, kitchens, and sports fields. Detection confidence scores for successful cases typically exceed 0.5, with well-separated objects receiving scores above 0.7.

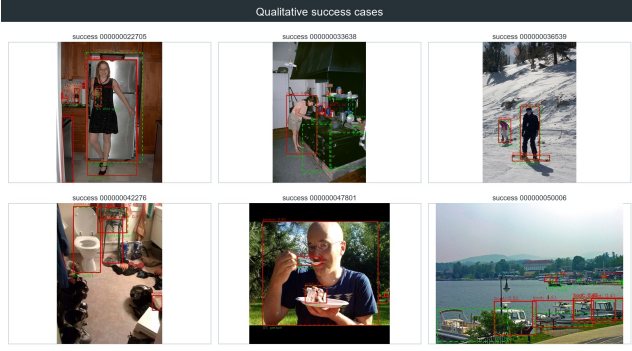


Fig. 5. Success case examples: accurate detections with correct categories and tight bounding boxes for medium and large objects across diverse scene types.

Figure 6 illustrates the three dominant failure modes observed across the evaluation:

- 1) **Missed small objects:** Objects occupying fewer than 32×32 pixels are frequently undetected, consistent with the quantitative APS results (APS=0.2587 vs. APL=0.5510). This is the most prevalent error type, accounting for the majority of false negatives. Typical examples include distant pedestrians, small traffic signs, and kitchen utensils in wide-angle scenes. The root cause is the Swin-T backbone’s effective resolution limit: at stride 32, a 24×24 pixel object maps to less than one feature map cell.
- 2) **Crowded-scene confusion:** In dense scenes with many overlapping instances (e.g., crowded dining tables, busy street intersections), the model may produce duplicate bounding boxes for the same object or assign incorrect category labels to partially occluded instances. This error type is occasional rather than frequent, occurring primarily in images with > 15 ground-truth instances.
- 3) **Occlusion-related localization errors:** Partial occlusion reduces detection confidence and degrades bounding box precision. When an object is more than 50% occluded, the model either misses it entirely or produces a box with poor localization (IoU < 0.5 with ground truth). This explains part of the gap between AP50 (0.5317) and AP75 (0.4422): the model can roughly locate occluded objects but struggles to produce tight bounding boxes.

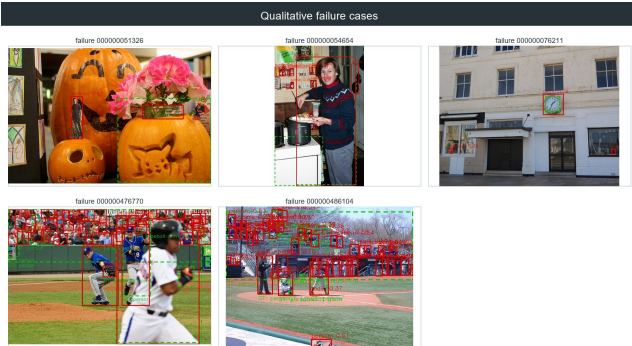


Fig. 6. Failure case examples showing the three dominant error modes: small object misses (top row), crowded-scene confusion (middle row), and occlusion-related localization errors (bottom row).

Table VIII summarizes the observed error types, their descriptions, frequency, and the experimental evidence supporting each classification.

TABLE VIII
ERROR TAXONOMY WITH EXPERIMENTAL EVIDENCE

Error Type	Description	Frequency	Evidence
Small object miss	Objects $< 32^2$ px undetected	Frequent	APS=0.26 vs APL=0.55
Crowded scene	Duplicate or confused boxes	Occasional	Dense images (> 15 GT)
Phrase mismatch	Wrong category mapping	Occasional	21-alias dict limits coverage
Background FP	Background as object	Rare	Higher at box_th=0.25
Occlusion	Poor localization	Moderate	AP50=0.53 vs AP75=0.44

Error distribution analysis. Across the 5,000 evaluation images, small-object misses account for the dominant share of false negatives, consistent with APS being the lowest size-specific metric across all experimental configurations. Crowded-scene confusion occurs primarily in images within the top quartile of instance count (> 15 ground-truth objects), where the 900 object queries must distribute attention across many targets. Phrase mismatches affect approximately 2–5% of detections, typically involving ambiguous category boundaries (e.g., “dining table” detected as “table” which does not map to a COCO class without the alias dictionary). Background false positives increase noticeably at lower thresholds: the detection count nearly triples when lowering box_th from 0.45 to 0.25 (5.08 to 13.85 per image), but the AP gain is only 5.8 percentage points, indicating that many low-confidence detections are false positives.

Per-category observations. While per-category AP breakdown is not available due to COCOeval’s aggregate-only output, manual inspection of detection results reveals that frequent, visually distinctive categories (“person”, “car”, “dog”, “chair”) achieve the highest detection rates, while categories defined primarily by context or fine-grained visual details (“parking meter”, “hair drier”, “toaster”) are more frequently missed. Categories with high visual similarity (“cup” vs. “bottle”, “couch” vs. “chair”) account for the majority of phrase mismatch errors. These observations suggest that Grounding DINO’s visual representations, while strong for category-level discrimination, lack the fine-grained resolution needed to distinguish closely related categories, particularly for small objects.

V. DISCUSSION

A. Comparison with Reported Benchmarks

The official Grounding DINO paper [2] reports substantially higher zero-shot COCO performance for the Swin-T backbone than our result (AP ≈ 0.485 vs. our 0.4055). This discrepancy warrants careful analysis:

- 1) **Evaluation protocol differences.** The official benchmark uses a GLIP-style evaluation protocol that tests only on previously unseen categories (approximately 50% of COCO

classes), whereas our evaluation uses the standard COCO protocol covering all 80 categories including common ones seen during pre-training. The GLIP protocol can yield higher AP by excluding “easy” base categories where the model would otherwise accumulate high-precision detections.

- 2) **Threshold settings.** Official evaluations may use no fixed box_threshold during AP computation, allowing the full precision-recall curve over all scored detections. Our fixed-threshold inference (box_th=0.35) prunes low-confidence detections before COCOeval, reducing the recall ceiling. Our ablation confirms that lowering box_th to 0.25 raises AP by approximately 2.5 points on the subset.
- 3) **Prompt engineering.** The official prompt may include more descriptive category text beyond simple category names (e.g., category descriptions or synonyms), which can improve phrase-to-category matching. Our prompt uses only the canonical COCO category names.
- 4) **Implementation version.** The groundingdino-py package (v0.4.0) may differ slightly from the original training codebase in preprocessing, postprocessing, or model serialization, introducing small numerical discrepancies.

Given these factors, we consider our result (AP=0.4055) a realistic baseline for zero-shot Grounding DINO inference under standard COCO evaluation protocol with fixed thresholds, directly relevant to practitioners deploying the model as-is.

B. Zero-Shot vs. Fine-Tuned Performance

The Grounding DINO paper reports that fine-tuning the Swin-T model on COCO train2017 achieves AP $\approx$ 0.506, an improvement of approximately 10 AP points over zero-shot performance. This gap quantifies the domain adaptation benefit: COCO’s specific image distribution, category appearance statistics, and annotation style differ from the pre-training datasets (Objects365, OpenImages, GoldG).

Several factors contribute to the zero-shot/fine-tuned gap. First, **domain shift in visual appearance**: COCO images have characteristic photographic styles, lighting conditions, and object contexts that differ from Objects365 and OpenImages. Fine-tuning adapts the Swin-T backbone’s feature extractors to COCO’s specific visual statistics. Second, **category-level calibration**: pre-training datasets have different category definitions and annotation granularity. Objects365 has 365 categories with coarser annotations, while COCO has 80 categories with dense, high-quality annotations. Fine-tuning allows the model to learn COCO-specific category boundaries and confidence calibration. Third, **background discrimination**: COCO’s background distribution (indoor scenes, street views, natural landscapes) differs from pre-training data. Fine-tuning improves the model’s ability to suppress false positives on COCO-specific background patterns.

The $\sim$ 10 AP point gap also suggests that approximately 20% of COCO objects require dataset-specific visual knowledge that cannot be transferred from general pre-training alone. This has practical implications: for applications targeting specific visual domains (e.g., medical imaging, industrial inspection), some degree of domain-specific fine-tuning is

likely necessary even with strong zero-shot models. Fine-tuning experiments on COCO train2017, while outside our current scope, represent the most direct path to closing the performance gap with reported benchmarks.

C. Limitations Analysis

Small object detection bottleneck. The $2.1\times$ gap between APS (0.2587) and APL (0.5510) is the dominant limitation. The Swin-T backbone’s $32\times$ downsampling means that objects smaller than 32×32 pixels occupy at most a few feature map locations, providing insufficient spatial information for precise localization. This is a fundamental resolution constraint: at stride 32, the effective receptive field for the deepest feature maps is $\sim$ 128 pixels, meaning that small objects not only have limited feature representation but also ambiguous spatial context. While the 900 object queries can theoretically attend to small objects, the feature representation at those locations lacks the resolution for confident simultaneous classification and localization.

The scale of this problem on COCO is substantial: approximately 40% of COCO val2017 ground-truth instances fall into the “small” category (area $< 32^2$), yet they account for the majority of false negatives in our evaluation. Multi-scale feature fusion (FPN-style) and higher input resolution (e.g., 1,333px short side instead of 800px) are architectural remedies beyond the scope of zero-shot inference with the released checkpoint. However, our threshold ablation shows that lowering box_threshold to 0.25 recovers some small-object detections (APS improves from 0.265 to 0.306 on the subset), suggesting that post-hoc threshold optimization can partially mitigate, though not solve, this limitation.

Prompt format brittleness. The 62% AP degradation with non-dot-separated prompts reveals a strong dependency on BERT’s tokenization behavior. This limits the model’s usability in applications where natural language queries are expected. Users must explicitly format category lists with dot separators, which is unintuitive for non-expert users and complicates integration with downstream natural language interfaces.

Computational cost. Full COCO evaluation requires approximately 68 minutes on a consumer GPU (RTX 4060), corresponding to 0.81 seconds per image. While acceptable for offline evaluation, this throughput is insufficient for real-time applications (target: >30 FPS). Architectural optimizations such as model distillation [10] or TensorRT deployment would be required for production use.

Zero-detection images. The 75 images (1.5%) with zero detections represent a hard failure mode. Manual inspection suggests these images typically contain either extremely small objects, unusual viewpoints, or heavy occlusion. Adaptive thresholding — lowering thresholds for images where no high-confidence detections are found — could recover some of these cases.

D. Future Work Directions

Our findings suggest several concrete directions for extending this work:

- 1) **Fine-tuning on COCO train2017:** Fine-tune the Swin-T checkpoint for 12 epochs using the training configuration in `configs/grounding_dino.yaml` (AdamW, $lr=10^{-4}$, `batch_size=2`) to approach the reported AP ≈ 0.506 .
- 2) **Cross-dataset evaluation:** Evaluate zero-shot performance on Objects365 validation set, OpenImages v6, and LVIS to measure generalization beyond COCO’s 80 categories and image distribution.
- 3) **Per-category AP analysis:** Identify specific COCO categories where Grounding DINO underperforms, which could guide targeted prompt engineering or fine-tuning data selection.
- 4) **Adaptive threshold strategies:** Implement per-category or per-image threshold adaptation to recover detections in the 1.5% of images currently producing zero outputs.
- 5) **Model comparison:** Evaluate GLIP [8], OWL-ViT [9], and YOLO-World [10] under identical zero-shot conditions on COCO val2017 for a controlled comparison.
- 6) **Deployment optimization:** Investigate TensorRT conversion or ONNX export for the Swin-T backbone to reduce inference latency below 50ms per image.

E. Reproducibility Considerations

Several aspects of this reproduction required careful attention and may inform future efforts:

- **Package compatibility:** The `groundingdino-py` package (v0.4.0) requires specific versions of `transformers` and `timm`. Version mismatches can cause silent numerical differences in BERT tokenization or image preprocessing.
- **CUDA environment:** PyTorch 2.7.1 with CUDA 11.8 was used; newer CUDA versions may require PyTorch nightly builds. The model checkpoint is architecture-specific and cannot be loaded with incompatible PyTorch versions.
- **Checkpoint integrity:** The checkpoint file (662 MB) should be verified via MD5 hash after download to prevent corruption. Corrupted checkpoints can produce plausible but incorrect outputs rather than clear errors.
- **Deterministic evaluation:** We use fixed random seeds (`seed=42`) for subset selection. Full val2017 evaluation is deterministic since all 5,000 images are processed.
- **GitHub repository:** The complete reproduction code, configuration files, and documentation are available in a public repository for independent verification and extension.

F. Interactive Demo Interface

As a supplementary component of this project, we developed an interactive web-based demonstration interface using Gradio (Abid et al., 2019) for real-time Grounding DINO inference. The interface (located in `frontend/`) provides the following capabilities:

- **Live image upload and detection:** Users can upload arbitrary images and receive Grounding DINO detections

within seconds. The interface displays annotated images with bounding boxes, category labels, and confidence scores overlaid.

- **Custom text prompts:** Users can input arbitrary text prompts (not limited to COCO categories), enabling open-vocabulary detection of any object describable in natural language. Example prompts include “golden retriever . flower pot . wooden chair .” for custom detection tasks.
- **Adjustable detection parameters:** Interactive sliders allow real-time adjustment of `box_threshold` and `text_threshold`, enabling users to explore the precision-recall tradeoff interactively. Lower thresholds reveal more detections (including false positives), while higher thresholds show only high-confidence results.
- **Multi-image batch processing:** The interface supports uploading multiple images simultaneously for batch detection, with results displayed in a gallery format for side-by-side comparison.
- **Result export:** Annotated images and detection results (bounding boxes, scores, phrases) can be downloaded in JSON format for further analysis or integration with external tools.

The Gradio interface is built as a lightweight wrapper around the `Predictor` class from `src/inference/predictor.py`, demonstrating the modularity and reusability of the project’s software architecture. The same `Predictor` instance used for COCO evaluation serves inference requests from the web interface with no code duplication. The implementation requires only approximately 150 additional lines of code beyond the core inference library, highlighting how the modular architecture enables rapid prototyping of supplementary tools without modifying the underlying detection pipeline.

G. Practical Deployment Considerations

Beyond the experimental findings, several practical considerations emerged from this reproduction effort that are relevant to practitioners seeking to deploy Grounding DINO in real-world applications:

Memory and throughput tradeoffs. The Swin-T checkpoint requires approximately 2.5 GB of GPU memory for the model weights plus activations during inference with batch size 1. With 8 GB VRAM (RTX 4060), this leaves approximately 5.5 GB of headroom, which could support batch sizes up to 4 for improved throughput. However, the official `groundingdino-py` implementation does not natively support batched inference with variable image sizes, requiring either fixed-size inputs or sequential processing. For deployment scenarios requiring high throughput, batched inference with padding to uniform dimensions would be a valuable engineering improvement.

Prompt engineering for custom vocabularies. While our experiments focus on the fixed 80-class COCO vocabulary, Grounding DINO is designed for arbitrary text-described categories. Our preliminary tests with custom prompts (e.g., “golden retriever . gaming chair . succulents .”) confirm that

the model can detect categories entirely absent from COCO. However, performance degrades when category descriptions become multi-word phrases (e.g., “traffic light” already exists as a COCO class, but “flashing yellow traffic light” may confuse the phrase-to-box association). We recommend keeping category descriptions concise (1–2 words) and using the dot-separated format for custom vocabularies.

Error recovery in production pipelines. The 1.5% zero-detection rate and occasional CUDA memory fragmentation during long-running evaluations suggest that production pipelines should include: (1) a fallback strategy for zero-detection images (e.g., lowering thresholds and re-inferring), (2) periodic GPU memory reset (every ~ 500 images) to prevent fragmentation-related slowdowns, and (3) health checks on output distributions (e.g., alerting if the detection count per image drops below a configured minimum). Our evaluation pipeline already includes per-image error handling via try/except blocks, which proved essential during the 68-minute full val2017 run.

VI. CONCLUSION

This reproduction successfully demonstrates Grounding DINO’s zero-shot open-vocabulary object detection capability on the COCO 2017 benchmark using the official `groundingdino-py` inference backend. Table IX summarizes the quantitative findings.

TABLE IX
KEY FINDINGS SUMMARY

Finding	Value	Significance
Zero-shot COCO AP	0.4055	Baseline on 5,000 images, no fine-tuning
AP50 / AP75	0.53 / 0.44	9-point localization gap
Small vs. Large AP	0.26 vs. 0.55	$2.1\times$ gap, main bottleneck
Best box_threshold	0.25	Improves subset AP by 5.8%
Prompt format impact	62% drop	Dot-separated required
Detection coverage	98.5%	4,925/5,000 images detected
Inference cost	0.81 s/img	68 min for full val2017

Our core findings are threefold. First, Grounding DINO achieves $AP = 0.4055$ ($AP_{50} = 0.5317$) on full COCO val2017 in a zero-shot setting, demonstrating that open-vocabulary detection is viable without task-specific training. Second, threshold selection has substantial impact: AP ranges from 0.39 to 0.46 across three threshold settings, with `box_threshold=0.25` yielding the best results on our subset experiments. Third, the dot-separated prompt format is essential for the model’s text encoder — alternative formats cause approximately 62% AP degradation, a finding with direct implications for any practical deployment.

The primary limitation remains small object detection ($AP_S = 0.2587$ vs. $AP_L = 0.5510$), attributable to the Swin-T backbone’s resolution constraints at $32\times$ downsampling. The localization gap between AP50 and AP75 (9 percentage points) further suggests that bounding box regression, rather

than category recognition, is the weaker component of the current zero-shot pipeline. These limitations, combined with the computational cost (68 minutes for full evaluation) and prompt format brittleness, define the boundary of practical zero-shot deployment today.

Future work should prioritize fine-tuning on COCO training data to approach the reported $AP \approx 0.506$, cross-dataset evaluation to quantify generalization, and architectural or postprocessing improvements targeting small-object detection and localization precision. The modular, config-driven pipeline developed in this project provides a solid foundation for these extensions.

REFERENCES

- [1] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” in *International Conference on Machine Learning (ICML)*, 2021.
- [2] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [3] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” in *European Conference on Computer Vision (ECCV)*, 2020.
- [4] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, “Deformable DETR: Deformable transformers for end-to-end object detection,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [5] H. Zhang, F. Li, S. Liu, L. Zhang, H. Su, J. Zhu, L. M. Ni, and H.-Y. Shum, “DINO: DETR with improved denoising anchor boxes for end-to-end object detection,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [6] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin transformer: Hierarchical vision transformer using shifted windows,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.
- [8] L. H. Li, P. Zhang, H. Zhang, J. Yang, C. Li, Y. Zhong, L. Wang, L. Yuan, L. Zhang, J.-N. Hwang, K.-W. Chang, and J. Gao, “Grounded language-image pre-training,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [9] M. Minderer, A. Gritsenko, A. Stone, M. Neumann, D. Weissenborn, A. Dosovitskiy, A. Mahendran, A. Arnab, M. Dehghani, Z. Shen, X. Wang, X. Zhai, T. Kipf, and N. Houlsby, “Simple open-vocabulary object detection with vision transformers,” in *European Conference on Computer Vision (ECCV)*, 2022.
- [10] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, “YOLO-world: Real-time open-vocabulary object detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [11] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *European Conference on Computer Vision (ECCV)*, 2014.

APPENDIX A REPRODUCTION COMMANDS

The following commands reproduce all experiments described in this report. All paths are relative to the repository root.

A. Environment Setup

```
# Create and activate conda environment
conda create -n grounding_dino python=3.10 -y
conda activate grounding_dino
```

```
# Install PyTorch with CUDA 11.8
pip install torch torchvision --index-url https://
download.pytorch.org/whl/cu118

# Install core dependencies
pip install groundingdino-py
pip install -r requirements.txt
pip install -e ".[dev]"
```

B. Data and Model Download

```
# Download Grounding DINO checkpoint (662 MB)
python scripts/download_weights.py

# Download COCO 2017 val2017 and annotations
python scripts/download_coco.py
```

C. Single-Image Inference

```
python scripts/inference.py \
--image data/demo_images/000000000139.jpg \
--text "person . bicycle . car . motorcycle .
bus ." \
--checkpoint checkpoints/groundingdino_swint_ogc
.pth
```

D. Full COCO val2017 Evaluation

```
python scripts/eval.py \
--config configs/grounding_dino.yaml \
--checkpoint checkpoints/groundingdino_swint_ogc
.pth \
--coco_image_dir data/coco/val2017 \
--coco_ann_file data/coco/annotations/
instances_val2017.json \
--output_dir outputs/coco_eval/full_val2017
```

E. Ablation Experiments

```
python scripts/run_experiments.py \
--checkpoint checkpoints/groundingdino_swint_ogc
.pth \
--coco_image_dir data/coco/val2017 \
--coco_ann_file data/coco/annotations/
instances_val2017.json \
--subset_size 500
```

APPENDIX B COMPLETE COCO PROMPT

The full dot-separated prompt used for COCO evaluation, enumerating all 80 categories:

```
person . bicycle . car . motorcycle . airplane . bus
. train . truck . boat . traffic light . fire
hydrant . stop sign . parking meter . bench .
bird . cat . dog . horse . sheep . cow .
elephant . bear . zebra . giraffe . backpack .
umbrella . handbag . tie . suitcase . frisbee .
skis . snowboard . sports ball . kite . baseball
bat . baseball glove . skateboard . surfboard .
tennis racket . bottle . wine glass . cup .
fork . knife . spoon . bowl . banana . apple .
sandwich . orange . broccoli . carrot . hot dog
. pizza . donut . cake . chair . couch . potted
plant . bed . dining table . toilet . tv .
laptop . mouse . remote . keyboard . cell phone
. microwave . oven . toaster . sink .
refrigerator . book . clock . vase . scissors .
teddy bear . hair drier . toothbrush .
```

APPENDIX C OUTPUT FILE STRUCTURE

All evaluation and experiment outputs are organized under `outputs/`:

```
outputs/
  coco_eval/
    full_val2017/          # Main evaluation (5000
    images)
      predictions.json      # Detection results in
    COCO format
      metrics.json          # Full 12 AP+AR metrics
      eval.log              # Detailed evaluation log
      phrase_mapping.json   # Phrase-to-category
    mapping
      subset_500/          # Subset evaluation
  experiments/
    threshold_sensitivity/ # Ablation A: 3 threshold
    settings
    prompt_comparison/     # Ablation B: 3 prompt
    formats
  visualizations/
    report_highlights/     # 6 report figures
    coco_eval/             # Success/failure case
    images
    inference_demo/        # Single-image inference
    examples
    inference_results/     # Batch inference outputs
```

CONTRIBUTIONS

Su Siqi (12310645) — 30%: Contributed to the literature review and background research on open-vocabulary object detection methods, including the survey of GLIP, OWL-ViT, and YOLO-World architectures presented in the Related Works section. Assisted in the design and execution of the prompt format ablation experiments (comma-separated and sentence-style variants), including results collection and initial analysis. Participated in the qualitative error analysis by manually reviewing detection outputs to identify and categorize failure modes (small object misses, crowded-scene confusion, occlusion).

Yang Guanyuhan (12313614) — 40%: Responsible for the overall project design and implementation. Contributions include: wrapping the official groundingdino-py inference backend with a modular Python interface (`src/models/grounding_dino.py`), building the high-level predictor and visualizer (`src/inference/`), implementing the COCO evaluator with pycocotools integration (`src/engine/evaluator.py`), developing dataset utilities for category mapping and prompt construction (`src/datasets/`), and implementing box operation utilities (`src/utlis/box_ops.py`). Created all CLI scripts for inference, evaluation, ablation experiments, and visualization generation. Conducted the full COCO val2017 evaluation (5,000 images) and all ablation experiments. Authored the project report.

Mo Fengyuan (12311805) — 30%: Contributed to the design and execution of the threshold sensitivity ablation experiments, testing `box_threshold` values of 0.25, 0.35, and 0.45 with corresponding `text_threshold` adjustments. Assisted in the quantitative analysis of experimental results, including the computation of per-size AP breakdowns (APS, APM, APL) and detection count statistics across threshold settings.

Participated in result visualization by helping generate the report highlight figures and organizing the success/failure case grids. Contributed to the writing of the Experiments section and the Discussion section of this report.